

CS571 Programming Languages

Spring 2026

5. Imperative II: scopes



Agenda

1. Names and bindings
2. Storage allocation
3. Heap management (garbage collection)
4. **Scopes**

Scope

- The visibility or availability of names
 - Which code region is a name visible?
- When is it determined?
 - Static or Dynamic?
 - Lexical or Control-Flow?
- How & when does it change?

Static (Lexical) Scoping

- Scopes can be determined by the compiler
- All bindings for identifiers can be resolved by examining the program
- Typically, use the closest binding
- Used in most languages (C, Java, Scheme,...)

Nested Scope

- In Scheme, each (let ...) defines one scope
- In Python, Pascal, ALGOL, function can be nested
- In C, block scopes can be nested

```
(let ((x 0))  
  (let ((y 1)) (+ x y)))
```

```
def foo(x):  
    def bar(y):  
        return y*2  
    return bar(3) + x  
// bar not visible here
```

```
int foo (int x) {  
    {    int y=3; x=x+y; }  
    // y not visible here  
    return x; }
```



Nested Scope

- Closest Binding wins
- Shadows outer level of scope
- How are scope rules being checked?

Example (shadow.py)

 shadow.py > ...

```
1  x = 5
2  def f(x):
3      print(x)
4  def g(y):
5      x = y
6      print(x)
7  f(1)
8  g(2)
9  print(x)
```

In Python:

- `x = exp` (line 6): automatically introduces a new local variable `x`
- `nonlocal x`: makes `x` refer to the local variable `x` in the outer layer's nesting function.
- `global x`: makes `x` refer to the global variable `x`

Example (shadow.c)

C shadow.c >  main()

```
1  #include <stdio.h>
2  int x = 0;
3  void f(int x) {
4      printf("f:%d\n", x);}
5  void g(int y) {
6      x = y;
7      printf("g:%d\n", x);}
8  int main() {
9      int x = 5;
10     printf("main:%d\n", x);
11     f(1);
12     g(2);
13     printf("main2:%d\n", x);
14     return 0;}
```

In C:

- `void f(int x)` (line 3): variable introduced with type as argument is local
- `x = y` (line 6): variable used directly is referring to either global or outer level of nested scope
- `int x = 5` (line 9): variable introduced in function with type is local

Symbol Table

- Symbol table: a table of names and their bindings
- One table for each scope
 - Top level (Module/File/Global)
 - Function
 - Block (in C family languages)
- Nested scope:
 - A tree of connected symbol tables

Symbol Table

- Each entry in the symbol table contains
 - The name of an identifier
 - Additional information: its kind, its type, if it is constant and so on (compiler dependent)

Name	Kind	Type	Const?
x	id	int	0

Scope Check Example

```
(let ((x 0) (y 7))  
  (let ((y (+ x 1))  
        (x (+ y 1))))  
    (+ x y)))
```

(global)		
Name	Type	Constant?
(first let)		
Name	Type	Constant?
x	int	0
y	int	7
(second let)		
Name	Type	Constant?
y	int	1
x	int	8

Name Collision

```
(let ((x 0) (y 7))
  (let ((y (+ x 1))
        (x (+ y 1)))
    (+ x y)))
```

Which y?

Which y?

To find a binding:

1. Start from table of current scope
2. Go up until the name is found
3. Fail if no table contains the name

(global)		
Name	Type	Constant?
(first let)		
Name	Type	Constant?
x	int	0
y	int	7
(second let)		
Name	Type	Constant?
y	int	1
x	int	8



Name
Search

Scope Check Example

C symtab.c > ...

```
1  typedef int bool;
2  int x;
3  void f(int m){
4      float x, y;
5      {int i, j; x = 1;}
6      {int x, l; x = 2;}
7  }
8  int g(int n){
9      bool t;
10     x = 3;
11 }
```

In scope?

In scope?

In scope?

Scope Check Example

C symtab.c > ...

```

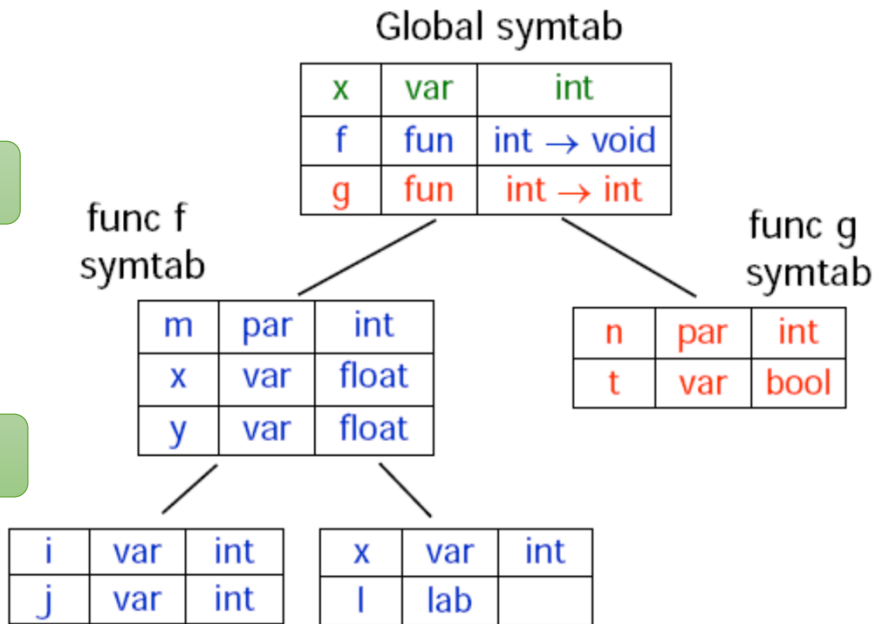
1  typedef int bool;
2  int x;
3  void f(int m){
4      float x, y;
5      {int i, j; x = 1;}
6      {int x, l; x = 2;}
7  }
8  int g(int n){
9      bool t;
10     x = 3;
11 }

```

In scope?

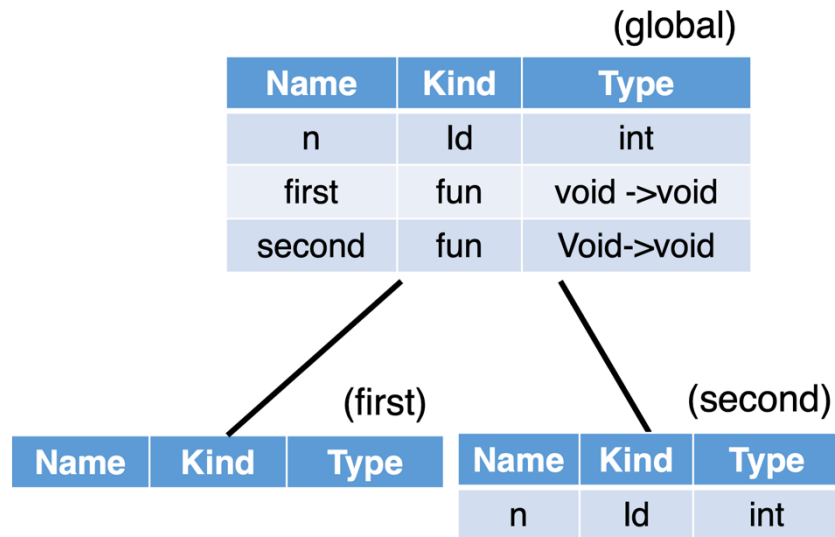
In scope?

In scope?



Static Scoping

```
int n = 0;
void first(){
    print(n);
    n = 1;
}
void second(){
    print(n);
    int n = 2;
    first();
    print(n);
}
first();
second();
```



- Symbol tables determined at compile time
- Search for the binding from the table for the current scope

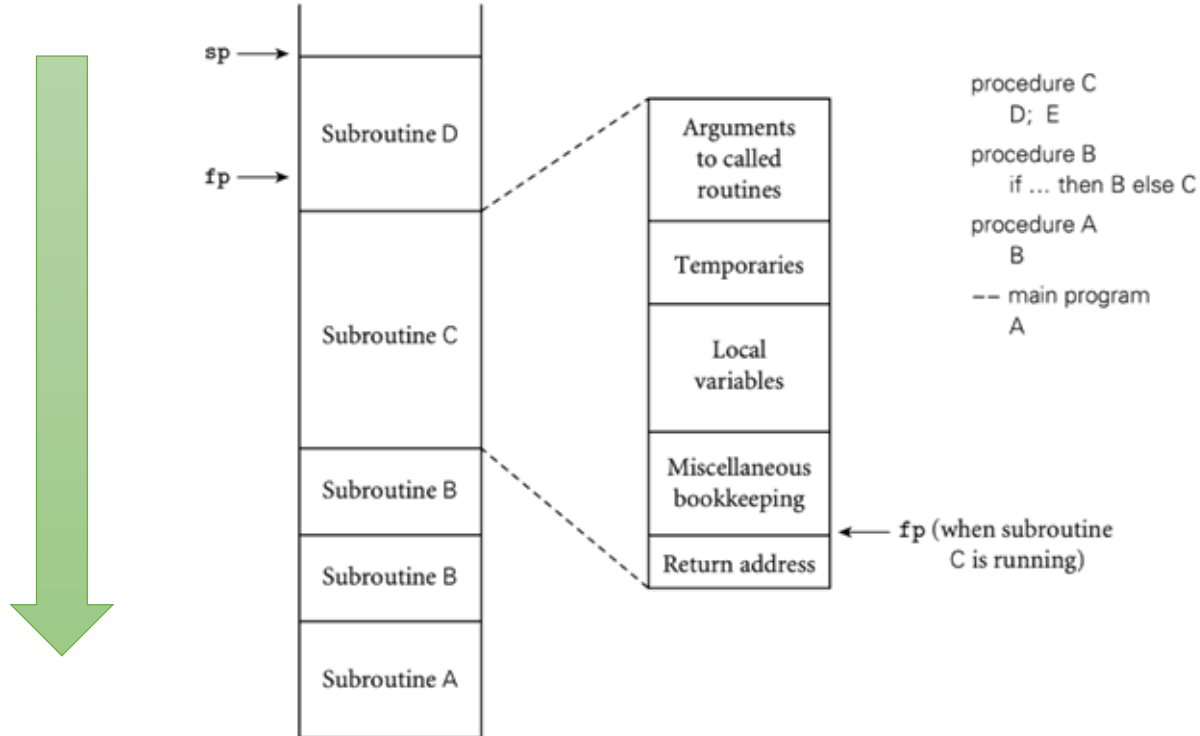


Dynamic Scoping

- Binding depend on runtime flow of control
- Always use the most recent active binding
- Hard to reason about code and variables by looking at source code alone.

Dynamic Scoping and Stack

Looking for
active
binding



Dynamic Scoping

```
int n = 0;
void first(){
    print(n);
    n = 1;
}
void second(){
    print(n);
    int n = 2;
    first();
    print(n);
}
first();
second();
```

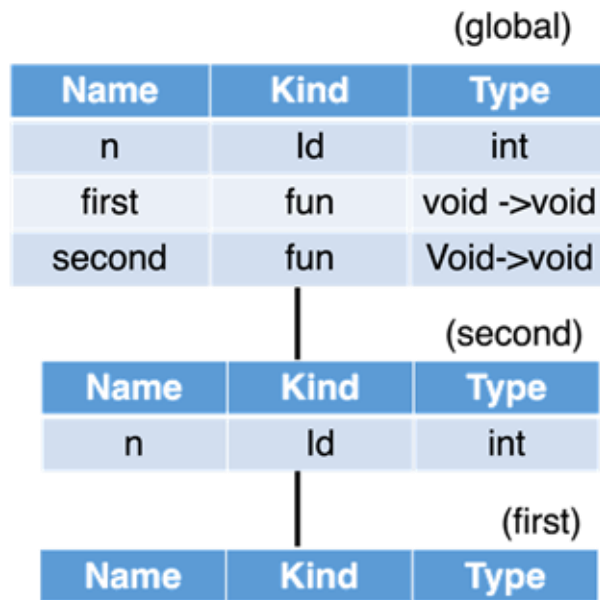
(global)		
Name	Kind	Type
n	ld	int
first	fun	void ->void
second	fun	Void->void

(first)		
Name	Kind	Type

Set global n to be 1

Dynamic Scoping

```
int n = 0;
void first(){
    print(n);
    n = 1;
}
void second(){
    print(n);
    int n = 2;
    first();
    print(n);
}
first();
second();
```



Set n in “second” to be 1

- Symbol tables changes at run time!
- Always use most recent, active binding



Bash Example

```
1  #!/usr/bin/env bash
2  n=0
3  √ first () {
4      echo "first:$n"
5      n=1
6  }
7  √ second () {
8      local n=2
9      echo "second:$n"
10     first
11     echo "second:$n"
12 }
13 first
14 second
15 echo "main:$n"
```

Output

```
first:0
second:2
first:2
second:1
main:1
```

Static vs Dynamic Scoping

- Similarity

- One symbol table per scope
- Names are resolved from bottom to top in symbol table tree

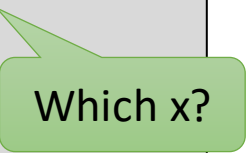
- Difference

- Static: the symbol table tree is stable
- Dynamic: the symbol table tree changes during execution

Function as a Parameter

- If a subroutine is passed as a parameter, how do we resolve names?
 - What's the parent of H's symbol table?

```
function F(int x) {  
    function G(fx) {  
        int x = 13;  
        fx();  
    };  
    function H() {  
        print x;  
    };  
    G(H);  
};
```



Which x?

Shallow vs Deep Binding

```
function F(int x) {  
    function G(fx) {  
        int x = 13;  
        fx();  
    };  
    function H() {  
        print x;  
    };  
    G(H);  
};
```

Shallow Binding: when
the subroutine is called

Deep Binding: when
reference is created

Shallow Binding

Use the environment at call time

```
function F(int x) {
    function G(fx) {
        int x = 13;
        fx();
    };
    function H() {
        print x;
    };
    G(H);
};
```

Established
at call time

(F)

Name	Kind	Type
x	para	int
G	fun	fun ->void
H	fun	void->void

(G)

Name	Kind	Type
fx	para	fun
x	ld	int

(H)

Name	Kind	Type
------	------	------

Deep Binding

Use the environment when reference is created

```
function F(int x) {  
    function G(fx) {  
        int x = 13;  
        fx();  
    };  
    function H() {  
        print x;  
    };  
    G(H);  
};
```

Established
when ref is
created

(F)

Name	Kind	Type
x	para	int
G	fun	fun ->void
H	fun	void->void

(H)

Name	Kind	Type
------	------	------

Deep Binding in Python

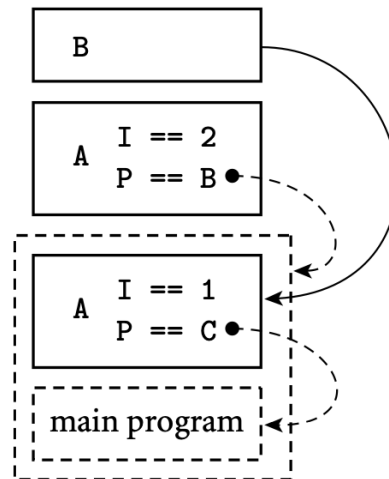
 dbinding.py >  A

```
1  def A(I, P):  
2      def B():  
3          print(I)  
4          if I>1:  
5              P()  
6          else:  
7              A(2, B)  
8  def C():  
9      pass  
10 A(1, C)
```

Deep Binding in Python

dbinding.py > A

```
1  def A(I, P):
2      def B():
3          print(I)
4          if I>1:
5              P()
6          else:
7              A(2, B)
8  def C():
9      pass
10 A(1, C)
```



Deep Binding in Python

 dbinding.py >  A

```
1  def A(I, P):
2      def B():
3          print(I)
4          if I>1:
5              P()
6          else:
7              A(2, B)
8  def C():
9      pass
10 A(1, C)
```

- Multiple Instances of A (and I in A) due to recursion
- Binding for I happened when B is used as an argument in the call A(2, B); not when B is called

Impl. of DB: Function Closures

```
function F(int x) {  
    function G(fx) {  
        int x = 13;  
        fx();  
    };  
    function H() {  
        print x;  
    };  
    G(H);  
};
```

Pass in a
function closure

- A **closure** is a pair: $\langle \text{code-ptr}, \text{env-ptr} \rangle$
- A function closure contains:
 - A pointer to the function
 - The current symbol table (or all symbol tables, depending on implementation)
- Can be represented using alists, with referencing environment pointing to alist giving environment when closure is created

Binding and Scoping

- Dynamic scoping
 - Both shallow and deep binding are implemented
 - Deep binding has a higher cost at run time
 - Most language uses shallow binding for ease of implementation
- Static scoping
 - Shallow binding has never been implemented

Class of Values

- First-class Values
 - Can be passed as a subroutine parameter, returned from a subroutine, stored in a variable, and optionally, be created at run time.
 - E.g.: integers and characters in most programming languages.
- Second-class Values
 - Can be passed as a subroutine parameter, but cannot be returned from a subroutine or assigned to a variable.
 - E.g.: subroutines in most programming languages.
- Third-class Values
 - Cannot be passed as subroutine parameter or returned by subroutine or assigned to a variable.
 - E.g.: labels in most programming languages.

Functions as First-Class Values

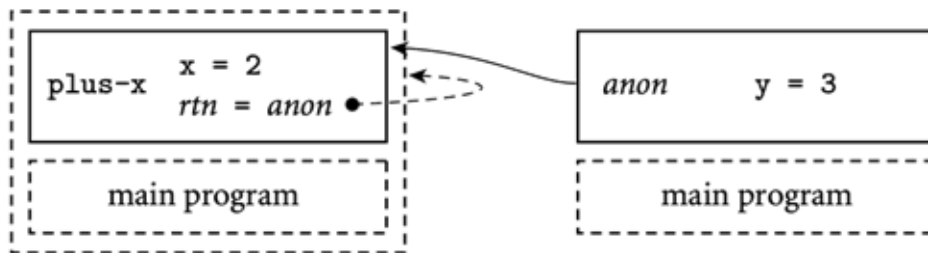
- If a function is returned from another function, then we need to preserve the referencing environment
- A language with first-class functions needs to have locals with unlimited extent (indefinite lifetime), conceptually allocated on the heap

Functions as First-Class Values

Functions are first-class values in all **functional programming languages** and most scripting languages

```
1. (define plus-x
2.   (lambda (x)
3.     (lambda (y) (+ x y))))
4. ...
5. (let ((f (plus-x 2)))
6.   (f 3))
```

; returns 5



Functions as First-Class Values

- Most imperative languages use stack-based allocation for local
- To avoid the dangling reference problem, they impose restrictions:
 - No nested routines for C, C++, Fortran
 - Modula-2 and Modula-3: only outermost subroutines are first-class values

Scope and Lifetime

- Scope refers to visibility of a binding/name
- Lifetime refers to creation/destruction of storage
- Scope and lifetime are usually connected
 - Global variable: static area, visible in entire program
 - Local variable: stack, visible in a function
- But not always!
 - Functions returned as values, heap allocated objects

Names and Objects

- So far, one-to-one mapping between names and visible objects
- In many languages, we also have:
 - Multiple names to same object
 - Multiple objects the same name

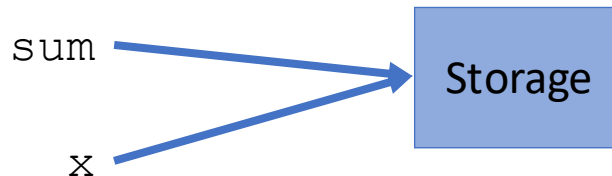
Alias

- Aliasing: multiple names referring to a single entity within a single scope
- Aliasing situations often occur implicitly in the presence of references and pointers

Alias

```
double sum, sum_of_squares;  
...  
void accumulate(double& x) {  
    sum += x;  
    sum_of_squares += x * x;  
}
```

x is passed by reference



Alias

```
int a, b, *p, *q;  
...  
a = *p;      /* read from the variable referred to by p */  
*q = 3;      /* assign to the variable referred to by q */  
b = *p;      /* read from the variable referred to by p */
```

What if *p and *q are aliases?



Function Overloading

- So far, two functions with same name cannot coexist (one hides the other)
- Overloading: uses the number or type of parameters to distinguish functions

Function Overloading

Symbol Table

```
void print (int a) {...};  
void print (bool a){...};  
void print (char a){...};  
print (3);
```

Name	Kind	Type
print	fun	int->void
print	fun	bool->void
print	fun	char->void

When check statement `print (3) :`

- All def. of “print” are returned
- The proper def. picked based on number or type of para.

Operator Overloading

```
class complex {
    double real, imaginary;
    ...
public:
    complex operator+(complex other) {
        return complex(real + other.real, imaginary + other.imaginary);
    }
    ...
};

...
complex A, B, C;
...
C = A + B;                // uses user-defined operator+
```

Coercion

Coercion: the process by which a compiler automatically converts a value of one type to a value with another type

Implicit

Explicit

```
void min (float a, float b) {...};  
min(1.0, 2.0);  
min(1, 2);  
min((float)1, (float)2);
```

Coercion vs Overloading

- Coercion converts parameters to fit function definition
- Overloading allows compiler to pick function implementation that fits

Coercion

```
void min (float a, float b)
min(1.0, 2.0);
min(1, 2);
```

Overloading

```
void min (float a, float b)
void min (int a, int b)
min(1.0, 2.0);
min(1, 2);
```



Coercion + Overloading can be Surprising

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  void evaluate(bool value) {
5      cout << "Evaluating a bool " << value << endl;
6  }
7  void evaluate(const std::string &value) {
8      cout << "Evaluating a string " << value << endl;
9  }
10 int main() {
11     evaluate(std::string("Hello world"));
12     return 0;
13 }
```

[Explanation](#)

Polymorphism

- Polymorphism: function with multiple forms
- Parametric polymorphism (fun. with a set of types)
 - Explicit para. polymorphism (Templates in C++)

```
template <class T>
T GetMax (T a, T b) {
    T result;
    result = (a > b)? a : b;
    return (result);
}
```

Polymorphism

- Polymorphism: function with multiple forms
- Parametric polymorphism (fun. with a set of types)
 - Explicit para. polymorphism (Templates in C++)
 - Implicit para. polymorphism (Lisp, Scheme, ML)

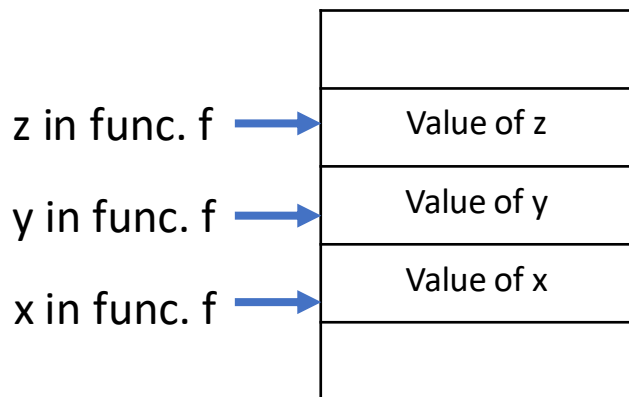
```
(define (min a b) (if (< a b) a b))
```

```
min a b = if a < b then a else b
```

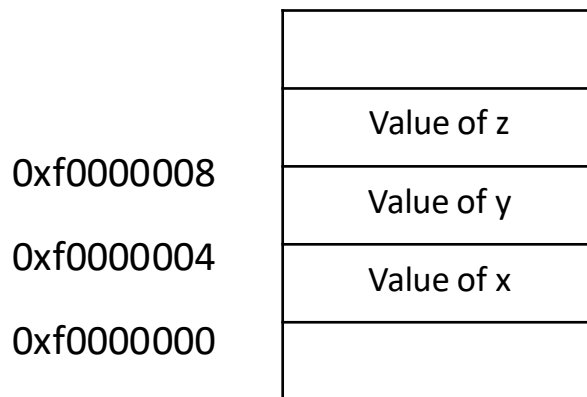
No mention
of types

Scope Implementation

How does the compiled code locate a variable used in a function at run time?



Memory abstraction in PL



Real memory

Establishing Addressability

How does the compiled code locate a variable used in a function at run time?

- What is the start address?
- Start address = **base address** (of a scope) + **offset** (in the scope)
 - Global variables
 - Local variables (w/ or w/o surrounding scopes)
 - Heap variables

Typical Code and Data Layout

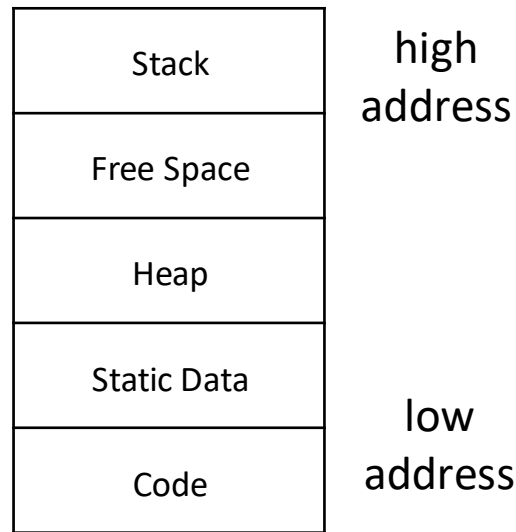
When executed, a program occupies memory

Code section

- Stores source code
- Read-only

Data section

- Static data area
- Stack
- Heap



Global & Static Variables

- Base address: fixed throughout the run
- Offset: known at compile time (length of each variable is determined by its type)

```
//Global variables  
int x;  
float y;  
int z;
```

Base + 12 (z)



Base + 4 (y)



Base (x)



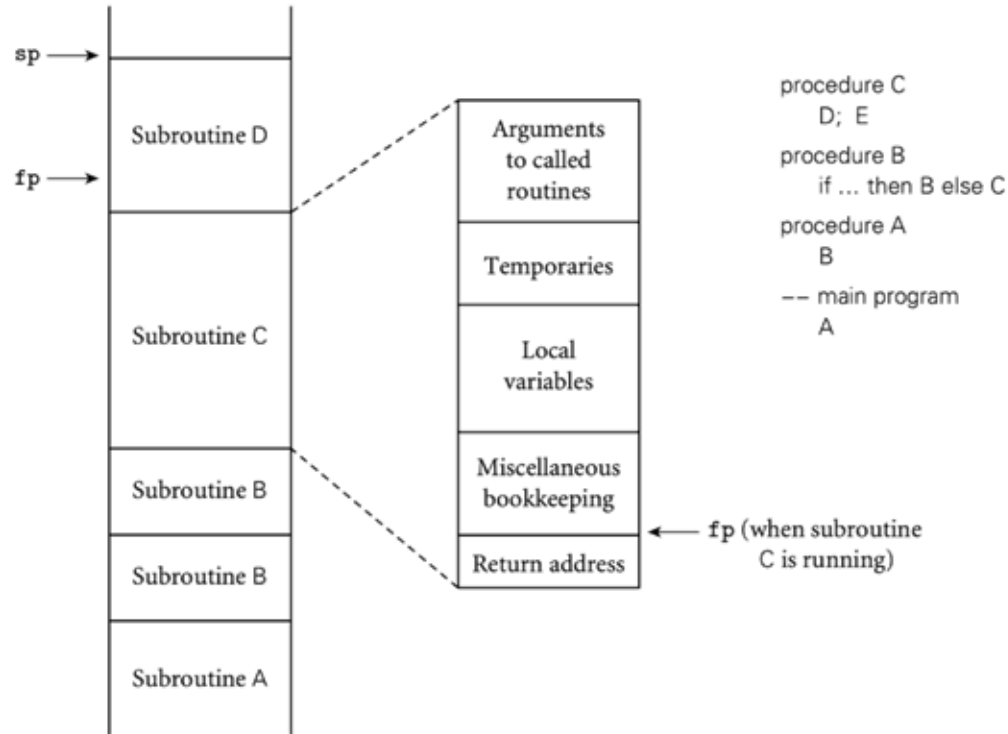
Static Data

Value of z

Value of y

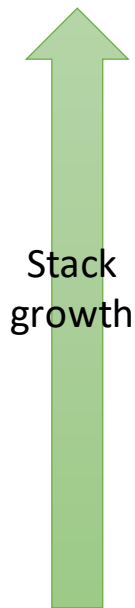
Value of x

Scope & Stack



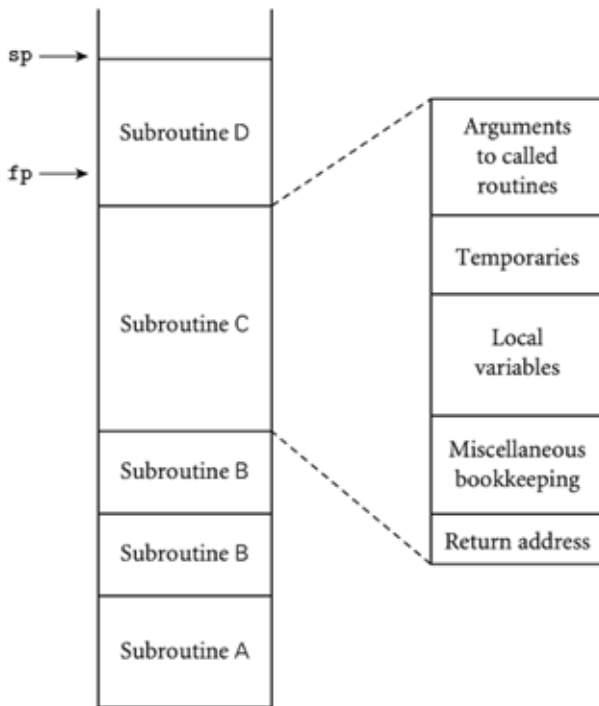
Scope & Stack

Low address



Stack
growth

High address



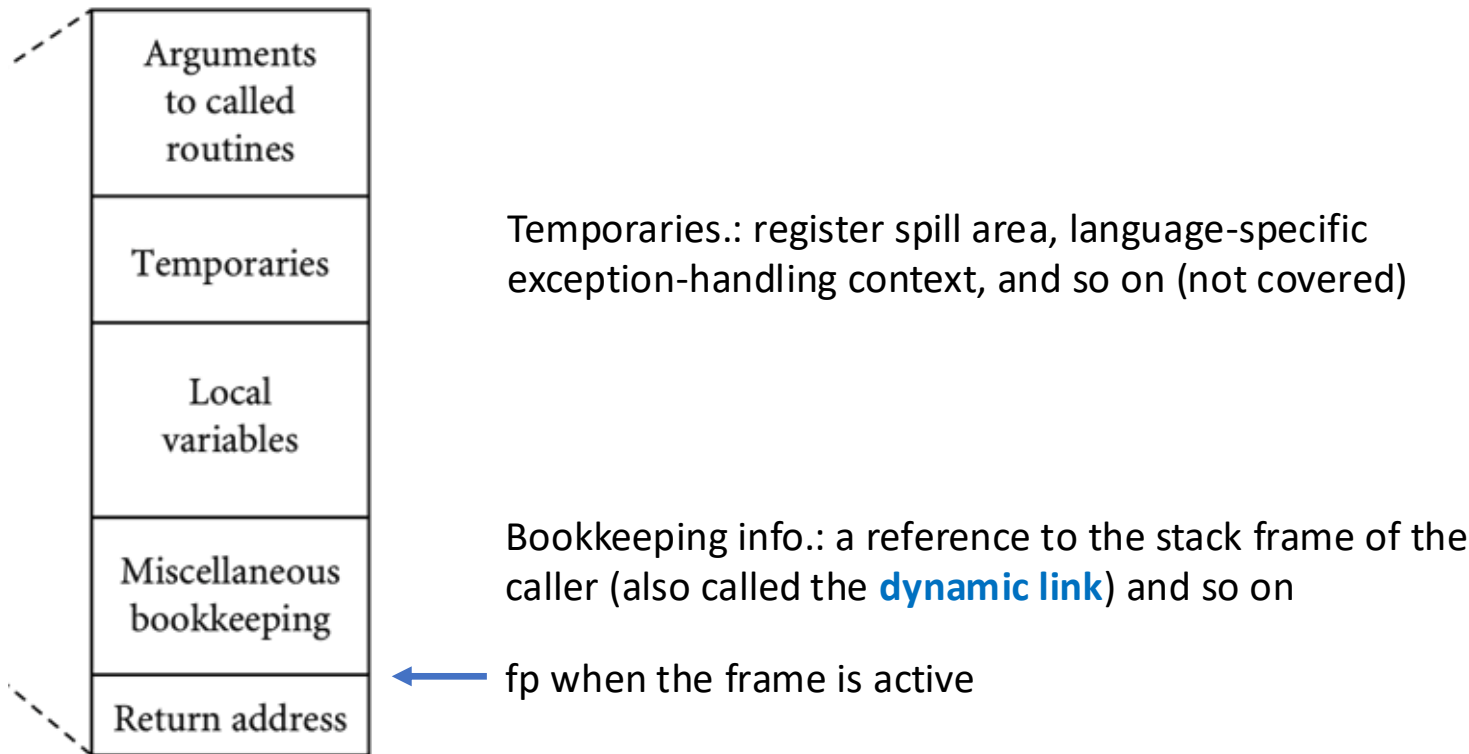
Each instance of a subroutine has a **frame (activation record)** at run time

- Compiler generates code that setup frame, call routine, and destroy frame
- **Frame pointer (fp)**: currently active frame
- **Stack pointer (sp)**: free space on stack

Hardware Support

- X86 Registers and instructions:
 - Stack pointer register: `esp`
 - Frame pointer register: `ebp`
 - Push instructions: `push`, `pusha`, etc.
 - Pop instructions: `pop`, `popa`, etc
 - Call instruction: `call`
 - Return instruction: `ret`

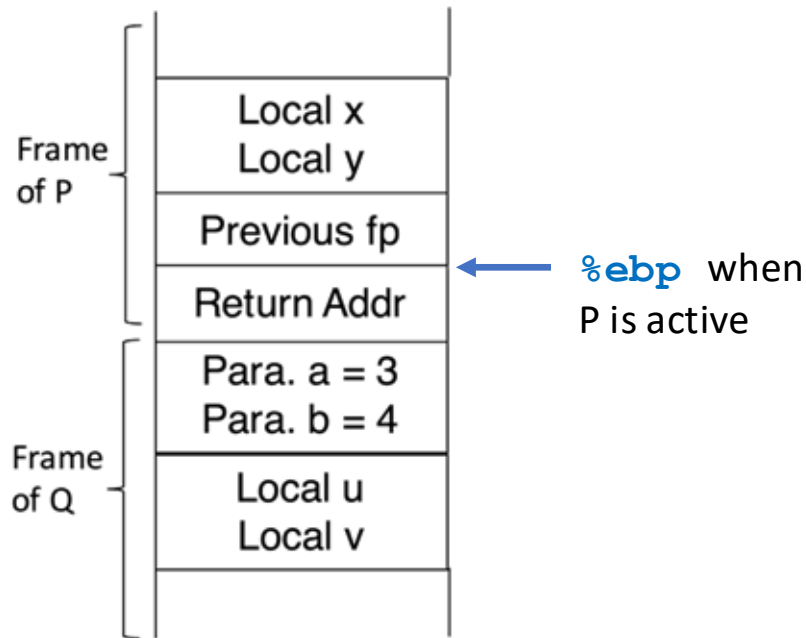
Typical Frame Layout



Local Variables

The offsets of objects within a frame usually can be statically determined

```
void P (int a, int b) {  
    int x, y;  
    ...  
}  
  
void Q () {  
    int u, v;  
    P (3, 4);  
}
```



Handling Nested Scope

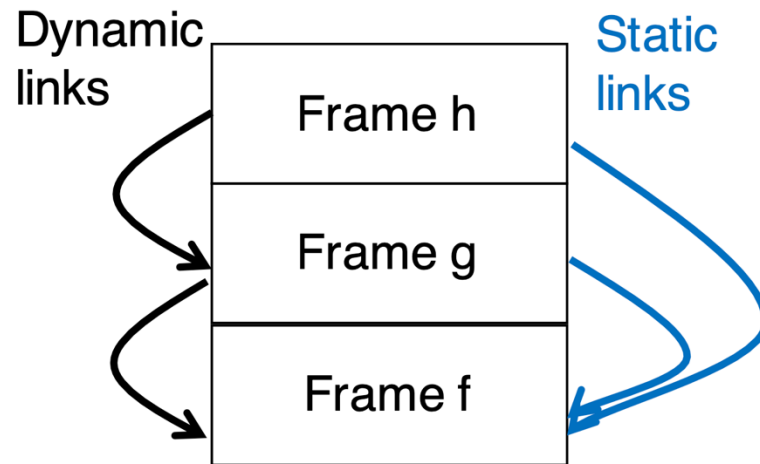
- Languages with nested functions
 - How do we access local variables from other frames?
- Static Scoping:
 - **Static link**: a pointer to the frame of enclosing function
- Dynamic Scoping
 - **Dynamic link**: pointer to the previous frame in the current execution (the same as previous fp)

Static Link and Dynamic Link

```
void f (int i) {  
    int a;  
    void h (int j) {  
        a = j;  
    }  
    void g (int k) {  
        h(k);  
    }  
    g(i+2);  
}
```

Static Link and Dynamic Link

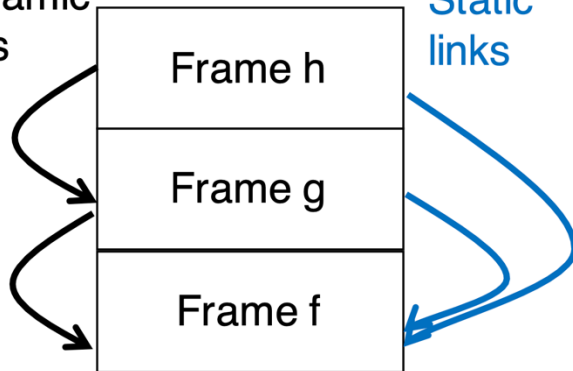
```
void f (int i) {  
    int a;  
    void h (int j) {  
        a = j;  
    }  
    void g (int k) {  
        h(k);  
    }  
    g(i+2);  
}
```



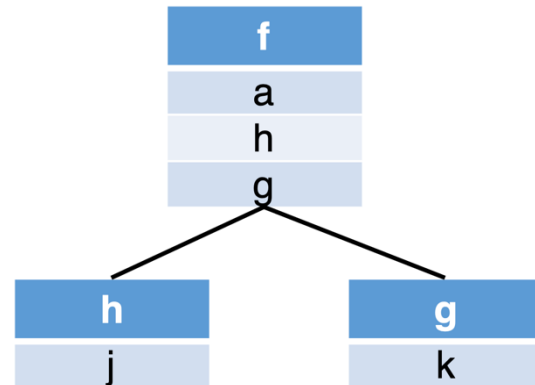
Where is a? (Static Scoping)

```
void f (int i) {  
    int a;  
    void h (int j) {  
        a = j;  
    }  
    void g (int k) {  
        h(k);  
    }  
    g(i+2);  
}
```

Dynamic
links



From frame of h, go up
one hop following **static**
links

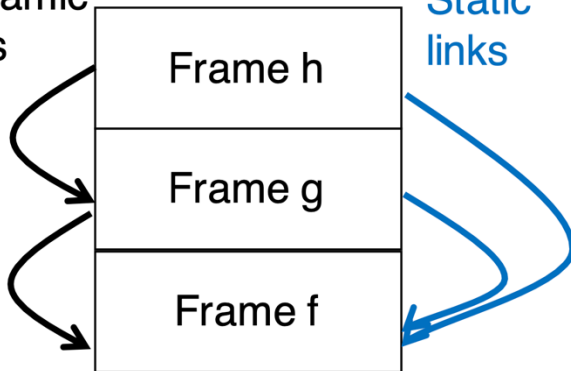


From symbol table of h,
go up **one hop**

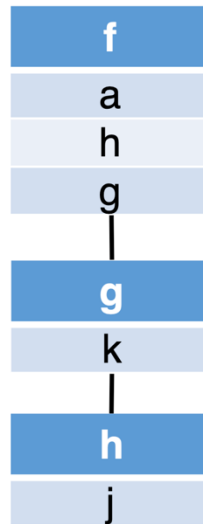
Where is a? (Dynamic Scoping)

```
void f (int i) {  
    int a;  
    void h (int j) {  
        a = j;  
    }  
    void g (int k) {  
        h(k);  
    }  
    g(i+2);  
}
```

Dynamic
links



Static
links

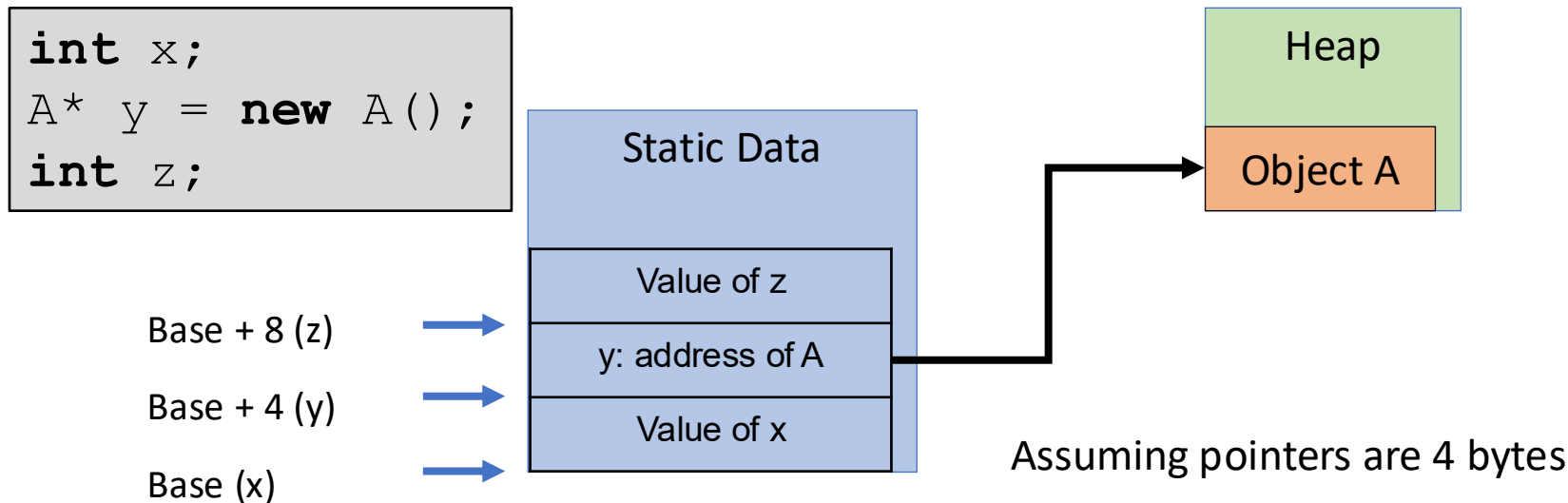


From frame of h, go up **two hops** following **dynamic links**

From symbol table of h, go up **two hops**

Heap Allocated Objects

Variables holding pointers to heap allocated objects are just like other variables. The variables themselves are global/local (y below) .



Big Picture: Data Memory Layout

